Lab 1D: Ship It

Duration: 35 min of material - in class you get 25 (3:15-3:40). Core: Phase 1, Phase 2, and Phase 4 through Step 5; Step 6 (push + PR) can be finished tonight.

After: Commands, Cost, Permissions & Git slides (Day 1, Block 4)

Prerequisites: Lab 1C complete (modular routes)

Objective

Cycle all four permission modes with Shift+Tab (including **Auto mode**), hand-write a project `.claude/settings.json` that ends the approval fatigue you felt in Lab 1A, prove the **deny -> ask -> allow** evaluation order by testing all three, then ship a feature through a real git branch -> PR workflow.

Mode names differ by surface - know both:

VS Code panel	Terminal	Behavior
Manual	Default	Asks before every edit
Edit automatically	AcceptEdits	Edits without asking
Plan	Plan	Explores and proposes; changes nothing
Auto	Auto	Approves what passes a safety check, **pauses for anything risky**
Bypass permissions	*(bypassPermissions)*	Stops asking, including before dangerous commands

`Shift+Tab` cycles them. **Auto and Bypass are not the same:** Auto has a safety check and pauses at the edge; Bypass removes the asking entirely. Treat Bypass like `sudo`.

> ***Surface note:*** *these keyboard steps assume* `claude` *is running in the* ***VS Code integrated terminal*** *(our classroom setup from Lab 0). If you are in the extension's chat panel instead, use its own mode switcher and rewind button - same capability, different control.*

Deliverable (toolkit chain 4 of 12)

Toolkit repo containing a **hand-written, committed** `.claude/settings.json` with allow/ask/deny rules you can defend, plus a CSV export feature and a pull request - your first end-to-end ship.

> ***START MARKER:*** `/clear` *run; dashboard serving on port 3100.*

Phase 1: Permission Modes - Shift+Tab (7 min)

Press **Shift+Tab** repeatedly and watch the mode indicator cycle:

Default -> AcceptEdits -> Plan -> Auto -> (back to Default)

Mode	Behavior	Use when
Default	Asks before edits and commands	Normal work
AcceptEdits	Auto-approves file edits; still asks for risky commands	Trusted repetitive edits
Plan	Read-only: Claude may plan but not change anything	Architecture decisions, risky areas
Auto	Works autonomously within your permission rules	Well-scoped tasks with guardrails

Step 1 - cycle to **Plan** mode and ask:

```
Add a simple bar chart to the dashboard showing task counts by status. Plan only.
```

Expected output marker: a plan, and NO files changed.

Step 2 - cycle to **Auto** mode and let it execute the plan:

```
Implement the chart plan now.
```

Expected output marker: Claude builds the chart without permission prompts. Refresh http://localhost:3100 - chart visible.

> ***Auto mode safety (Jun-Jul 2026):*** *Auto mode ships with two-layer protection - an input injection probe and an output transcript classifier. It blocks transcript tampering and asks before running* `rm -rf` *on unresolved variables. Autonomy comes fenced, and YOUR permission rules (next phase) are the fence.*

Step 3 - Shift+Tab back to **Default** for the rest of the lab.

Phase 2: Write Your Own Permission Rules - deny -> ask -> allow (12 min)

This is the fix for the approval fatigue you felt in Lab 1A

> *Remember Lab 1A - dozens of approval prompts, and by the end you were clicking without reading. That is the failure mode: friction so constant that you stop being a reviewer and become a rubber stamp.*
>
> *The fix is not "approve everything." The fix is to* ***decide once, in writing, which commands are safe*** *- and save your attention for the ones that aren't. That is what you are about to write. In your Lab 1A debrief you said which prompts you never wanted to see again; encode exactly those.*

Step 1 - look at the menu, then write the file yourself.

`/permissions` opens an interactive editor and is fine for a quick one-off. Open it once to see the three buckets:

```
/permissions
```

But the thing that actually ships with your project is a file. **Create** `.claude/settings.json` **in your toolkit repo yourself** - hand-written, not generated - so you own every line and can review it in a PR:

```
mkdir -p .claude
```

Create `.claude/settings.json` with an allow list for the commands **this course actually uses**, plus at least one deny rule and one ask rule:

```
{
"permissions": {
"deny": [
"Bash(rm -rf:*)",
"Bash(sudo:*)"
],
"ask": [
"Bash(git push:*)"
],
"allow": [
"Read",
"Grep",
"Glob",
"Bash(npm install:*)",
"Bash(npm test:*)",
"Bash(npm run:*)",
"Bash(node:*)",
"Bash(git status:*)",
"Bash(git diff:*)",
"Bash(git log:*)",
"Bash(git add:*)",
"Bash(git branch:*)",
"Bash(git checkout:*)"
]
}
}
```

Notes on syntax while you type it:

- `Bash(command:*)` is a **prefix match**, not a wildcard search.
- So `Bash(npm test:*)` covers `npm test` and `npm test -- --watch` - but NOT `npm run test`. List what you actually run.
- `Read` / `Grep` / `Glob` are tool names with no argument pattern - no parentheses.
- Project rules live in `.claude/settings.json` (committed, shared with the team). Personal overrides go in `.claude/settings.local.json` (git-ignored). Anything you want a reviewer to see belongs in the committed file.

Step 2 - restart and confirm Claude loaded it:

```
Show me the permissions section of .claude/settings.json and explain which rule wins if a
command matches both an allow and a deny pattern.
```

Expected output marker: Claude reads back your three arrays and states that **deny is evaluated first**, then ask, then allow - a deny wins even when an allow also matches. That ordering is what makes an allow list safe to write generously.

Step 3 - test all three paths. Do not skip any of them.

Allowed - should run silently, no prompt:

```
Read server.js and show me the first 10 lines, then run git status and git log --oneline -5
```

Expected: executes with **no approval prompt**. This is the Lab 1A friction, gone.

> *Why not `npm test` here? This dashboard has no `test` script - Lab 1A never generated one, so `npm test` would fail with Missing script: test and muddy the result. Keep `Bash(npm test:*)` in your allow list for the repos that do have tests; prove the allow path with commands this repo can actually run.*

Ask - should still prompt:

```
Push the current branch to origin
```

Expected: Claude **asks for approval** before pushing, even though other git commands run freely. You chose to keep this one in front of a human.

Denied - should refuse:

```
Run: rm -rf ./node_modules
```

Expected: **blocked**. Claude explains it cannot run the command and does not offer to approve it. Note the difference from "ask": there is no yes button.

Step 4 - commit it. This file is a deliverable.

```
Stage and commit .claude/settings.json with the message "chore: add project permission
rules"
```

Expected output marker: `.claude/settings.json` is committed to your toolkit repo. It is now shared configuration - every teammate who clones this repo inherits the same fence, and any change to it shows up in code review like any other change.

Phase 3: Price a Session (2 min)

```
/cost
```

Expected output marker: token usage and cost for today. Note the number.

> *First identify the billing path: subscription allowance versus API/cloud token billing. Use `/usage` for the live account. Rule of thumb: Sonnet 5 for normal implementation, Opus 5 for complex reasoning, and explicit `--max-turns` plus narrow tools for automation.*

Phase 4: Feature Branch -> PR (13 min)

Step 1 - branch:

```
Create a new git branch called "feature/export" and switch to it
```

Step 2 - build:

```
Implement a CSV export feature with these requirements:
- GET /api/export/metrics?format=csv - downloadable CSV of all metrics
- GET /api/export/tasks?format=csv - downloadable CSV of all tasks
- CSV logic must live in a reusable module (src/utils/csvExport.js)
- Routes remain thin controllers
- Add "Export CSV" buttons to the dashboard UI for both sections
- Follow CLAUDE.md conventions
```

Step 3 - test:

```
Test both CSV export endpoints and show me the output
```

Also click both Export buttons in the browser - each downloads a `.csv`.

Step 4 - agent-assisted diff review (before you commit - always):

```
Stage the export feature, then review the staged diff like a skeptical senior engineer: flag
risky changes, missed edge cases, and anything unrelated to this task. Do not commit yet.
```

Expected output marker: a short review - typically catching a stray debug line, an unrelated file that drifted in, or a missing error path. Fix anything real before moving on.

> *Review **before** commit, always. The agent's review is a second pair of eyes, not the approver - skim the diff yourself too.*

Step 5 - have Claude write the commit message:

```
Commit the export feature. Write the commit message yourself from the staged diff:
conventional format, what changed and why.
```

Expected output marker: a conventional commit whose body actually describes the diff - not "fixes stuff".

Step 6 - have Claude write the PR description and ship it. If you have a GitHub remote (from Lab 0 or `gh auth login`):

```
Push this branch and open a pull request titled "feat: CSV export for metrics and tasks".
Write the description yourself from the commits: summary, key changes, and how to test.
```

Expected output marker: Claude runs `git push` and `gh pr create`, returning a PR URL - and the PR body reads like a reviewer wrote it.

No GitHub? Merge locally instead:

```
Switch back to main and merge the feature branch.
```

Step 7 - review the history:

```
Show me the git log with a nice one-line format
```

Expected output marker: clean conventional commits telling the project's story.

FINISH MARKER - Success Checklist

- [ ] Cycled all four modes with Shift+Tab and used Plan then Auto deliberately
- [ ] `.claude/settings.json` **written by hand** in the toolkit repo, with an allow list covering the npm/node/git commands this course uses, at least one `ask` rule, and at least one `deny` rule
- [ ] **All three paths tested:** an allowed command ran with no prompt, the ask rule prompted, the deny rule refused
- [ ] Stated the deny -> ask -> allow evaluation order
- [ ] `.claude/settings.json` **committed to the toolkit repo** (it is a shared, reviewable deliverable - not a local tweak)
- [ ] You can name one Lab 1A prompt that this file eliminated, and one you deliberately kept
- [ ] `/cost` checked - you can price your day
- [ ] CSV export built on a feature branch
- [ ] Agent-assisted diff review run before committing - findings addressed
- [ ] Commit message and PR description written by Claude from the diff/commits
- [ ] PR opened (or branch merged locally)
- [ ] Clean conventional git history

If Stuck

Problem	Recovery
Shift+Tab does nothing	Click into the Claude input box first; in VS Code check the keybinding via `/keybindings`
Auto mode still asks	Some actions fall outside your allow rules - that's the design; add a rule or approve
Deny rule not blocking	Check exact syntax in `/permissions` - patterns are case-sensitive
settings.json ignored entirely	Invalid JSON (trailing comma is the usual culprit) or wrong path - it must be `.claude/settings.json` in the **project root**. Ask Claude: "Is my .claude/settings.json valid JSON and is it loaded?"
Allowed command still prompts	Your pattern doesn't match what actually ran - `Bash(npm test:*)` does not cover `npm run test`. Check the exact command in the prompt and widen the rule
`gh` not authenticated	Run `!gh auth login` or fall back to local merge
Chart/export broke the UI	Double-Esc, rewind code to the last good checkpoint, retry with a tighter prompt

Debrief Questions

1. When is Auto mode appropriate - and what makes it safe enough to use?
2. Why must deny be evaluated before allow?
3. Compare this lab to Lab 1A: which approvals disappeared, and which one did you deliberately keep in front of a human? Why that one?
4. Your `.claude/settings.json` is committed and shared. What would you want a teammate to challenge in code review before it merges?
5. What did the agent's diff review catch that you would have missed?
6. What was in your PR description that a reviewer would thank you for?